

TextSus

Code Documentation: How the Implementation Works

An implementation of SynthID-Text and its baselines

Dathathri, S., See, A., Ghaisas, S. et al. Scalable watermarking for identifying large language model outputs.
Nature 634, 818–823 (2024)

Repository: <https://github.com/Sparrowspidey/TextSus>

Paper: <https://doi.org/10.1038/s41586-024-08025-4>

Team:

- Himasai Vihar Suraboina 2024BCS0225
- Kurri Hiranya Venkata Reddy 2024BCS0221
 - Vivek Ediga 2024BCS0229

This document explains how the TextSus codebase implements LLM text watermarking: embedding a statistical signature into generated text during sampling, then detecting it later without needing the original model. It walks through every core module -- seed generation, g-values, Tournament sampling (both the paper's non-distortionary and distortionary configurations), the Gumbel and Soft Red List baselines, scoring and detection, the generation pipeline that ties them together, the evaluation utilities, and the experiment scripts used to produce the project's results.

1. Architecture Overview

Watermarking an LLM's output has three moving parts, and the codebase is organized around exactly this split: a **random seed generator** that turns recent context into a reproducible seed, a **sampling algorithm** that uses that seed to bias which token gets chosen, and a **scoring function** that later recomputes the same bias signal from finished text to decide whether it was watermarked. Detection never needs the original model -- only the token sequence and the secret key.

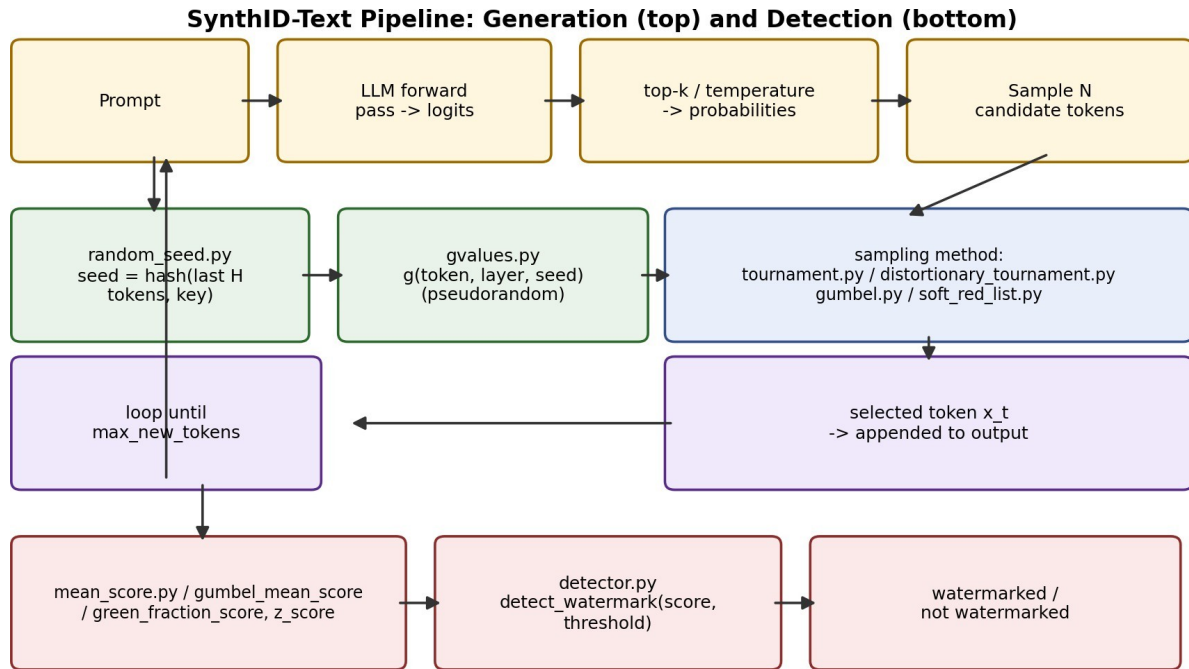


Figure 1: Generation (top three rows) feeds a sampled token back into the model for the next step; detection (bottom row) recomputes the same scores from a finished sequence of tokens, with no model access required.

1.1 Module Map

Path	Role
src/textsus/seed/random_seed.py	Sliding-window keyed hash -> per-step random seed
src/textsus/gvalues/gvalues.py	Pseudorandom g-value function (Bernoulli/Uniform)
src/textsus/sampling/tournament.py	Non-distortionary Tournament sampling (Algorithms 1-2)
src/textsus/sampling/distortionary_tournament.py	Gyeneralized Tournament, >2 competitors/match
src/textsus/sampling/gumbel.py	Gumbel (exponential-minimum) baseline sampling + scoring
src/textsus/sampling/soft_red_list.py	Soft Red List baseline: green-list logit bias + z-score
src/textsus/sampling/context_masking.py	Prevents re-watermarking a repeated context window
src/textsus/scoring/mean_score.py	Mean g-value detection score (paper's main scorer)
src/textsus/scoring/weighted_mean.py	Weighted variant of the mean score
src/textsus/scoring/bayesian.py	Bayesian/posterior-probability scoring function

src/textsus/scoring/frequentist.py	Frequentist hypothesis-test scoring (p-values)
src/textsus/detection/detector.py	Score -> watermarked/not-watermarked decision
src/textsus/generation/watermarked_generato	r.Tpiyes sampling methods into a HF generation loop
src/textsus/evaluation/metrics.py	TPR @ fixed FPR (the paper's detectability metric)
src/textsus/evaluation/perplexity.py	Perplexity, used as a text-quality proxy
src/textsus/evaluation/self_bleu.py	Self-BLEU, used as a text-diversity proxy
experiments/*.py	Runnable scripts that produce the project's results

2. Random Seed Generation

File: `src/textsus/seed/random_seed.py`. Every generation step needs a seed that is (a) reproducible from the text alone, so the detector can recompute it later, and (b) unpredictable without the secret key, so the watermark cannot be forged or stripped by an attacker who does not know the key. The implementation uses the paper's sliding-window method: hash the last H generated tokens together with the key.

```
H = 4

def generate_random_seed(tokens: list[int], key: int) -> int:
    window = tokens[-H:]
    data = ",".join(map(str, window)) + f"|{key}"
    digest = hashlib.sha256(data.encode("utf-8")).digest()
    return int.from_bytes(digest, byteorder="big")
```

Because SHA-256 is deterministic, calling this function with the same last-4-tokens and the same key always returns the same seed -- at generation time and at detection time -- with no need to store anything between the two.

3. g-values

File: `src/textsus/gvalues/gvalues.py`. The g-value function is the "referee" used throughout the rest of the pipeline. Given a candidate token, a tournament layer, and the seed for this step, it returns a pseudorandom score -- 0 or 1 under the default Bernoulli(0.5) distribution, or a float in $[0, 1)$ under the Uniform distribution. Critically, it is *deterministic*: the same (token, layer, seed) triple always hashes to the same value, so both the generator (choosing a token) and the detector (checking a token) compute identical numbers without ever communicating.

```
def generate_g_value(token_id, seed, layer, distribution="bernoulli"):
    data = f"{token_id}|{layer}|{seed}".encode("utf-8")
    digest = hashlib.sha256(data).digest()
    hash_value = int.from_bytes(digest, byteorder="big")
    uniform_value = hash_value / (2 ** (8 * len(digest)))
    if distribution == "bernoulli":
        return 1 if uniform_value >= 0.5 else 0
    if distribution == "uniform":
        return uniform_value
```

4. Tournament Sampling

4.1 Non-distortionary (2 competitors per match)

File: `src/textsus/sampling/tournament.py`. This is SynthID-Text's core contribution. Instead of sampling one token directly from the LLM's distribution, the method draws $N = 2^m$ candidate tokens (with replacement) and runs them through an m -layer single-elimination bracket. Each layer uses a different g-value function ($g_1, g_2, \dots, g_m$); within a pair, the token with the higher g-value at that layer wins, with ties broken uniformly at random. The last token standing is the output.

```
def single_layer_tournament_sample(token_ids, seed):
    g_values = [generate_g_value(t, seed, layer=1) for t in token_ids]
    max_g = max(g_values)
    winners = [t for t, g in zip(token_ids, g_values) if g == max_g]
    return random.choice(winners)

def tournament_sample(token_ids, seed, num_layers):
    candidates = token_ids.copy()
    for layer in range(1, num_layers + 1):
        winners = []
        for i in range(0, len(candidates), 2):
            a, b = candidates[i], candidates[i + 1]
            ga, gb = generate_g_value(a, seed, layer), generate_g_value(b, seed, layer)
            winner = a if ga > gb else b if gb > ga else random.choice([a, b])
            winners.append(winner)
        candidates = winners
    return candidates[0]
```

With exactly 2 competitors per match, this configuration is *single-token non-distortionary*: averaged over the random seed, the output token's distribution equals the LLM's original distribution, so text quality is preserved.

4.2 Distortionary (more than 2 competitors per match)

File: `src/textsus/sampling/distortionary_tournament.py`. The paper also defines a stronger, quality-costing configuration where each match has more than two competitors. This file generalizes the function above: at each layer, candidates are grouped into blocks of size *competitors_per_match*, and the single highest-g-value token in each block wins. With *competitors_per_match* = 2 it is mathematically identical to the function above; values above 2 bias token choice more aggressively toward high g-values, trading text quality for stronger, more easily detected watermarks.

```
def distortionary_tournament_sample(token_ids, seed, num_layers,
                                   competitors_per_match=2):
    candidates = token_ids.copy()
    for layer in range(1, num_layers + 1):
        winners = []
        for i in range(0, len(candidates), competitors_per_match):
            group = candidates[i:i + competitors_per_match]
            g_values = [generate_g_value(t, seed, layer) for t in group]
            max_g = max(g_values)
            best = [t for t, g in zip(group, g_values) if g == max_g]
            winners.append(random.choice(best))
        candidates = winners
    return candidates[0]
```

This module was added specifically to support the project's distortionary-vs-Soft-Red-List comparison experiment (Section 10.7), since the team's original tournament.py only implements the fixed 2-competitor case.

5. Baseline Sampling Methods

Two external watermarking schemes from prior work are implemented for comparison, sharing the same `random_seed.py` seed generator so the comparison with Tournament sampling is like-for-like, exactly as the paper's own evaluation does.

5.1 Gumbel Sampling (Aaronson & Kirchner)

File: `src/textsus/sampling/gumbel.py`. For every candidate token with LLM probability p_v , a pseudorandom value r_v is drawn from a keyed hash, and the token maximizing r_v^{1/p_v} is chosen. This reparameterization reproduces the original LLM distribution exactly on average (non-distortionary) while still leaving a detectable trace: the chosen token's r -value tends to have been large, specifically because that is why it won.

```
def gumbel_sample(probs: dict[int, float], seed: int) -> int:
    best_token, best_score = None, -1.0
    for token_id, p in probs.items():
        u = _uniform_hash(token_id, seed)
        score = u ** (1.0 / p)
        if score > best_score:
            best_token, best_score = token_id, score
    return best_token

def gumbel_token_score(token_id, seed):
    u = _uniform_hash(token_id, seed)
    return -math.log(max(1.0 - u, 1e-12))    # detection score
```

5.2 Soft Red List (Kirchenbauer et al.)

File: `src/textsus/sampling/soft_red_list.py`. At each step, the seed pseudorandomly splits the vocabulary into a "green list" (fraction *gamma*) and a "red list"; green-list logits are boosted by *delta* before sampling. This is inherently distortionary -- larger *delta* means a stronger, more detectable watermark at a larger quality cost, which is exactly the strength knob used in Section 10.7's comparison.

```
def is_green(token_id, seed, gamma=0.5):
    return _green_list_hash(token_id, seed) < gamma

def apply_soft_red_list_bias(logits, seed, gamma=0.5, delta=2.0):
    return {t: (1 + delta if is_green(t, seed, gamma) else 1)
            for t, l in logits.items()}

def z_score(token_ids, watermarking_key, gamma=0.5):
    n = len(token_ids)
    green_count = round(green_fraction_score(token_ids, watermarking_key, gamma) * n)
    expected, std = gamma * n, math.sqrt(n * gamma * (1 - gamma))
    return (green_count - expected) / std
```

Detection here checks whether a text's green-token fraction is far above what random chance (*gamma*) would predict, using a standard z-test -- unwatermarked text should show a green fraction near *gamma* and a

z-score near 0, which is exactly what the *delta*=0 sanity check in the project's results confirmed.

6. Repeated Context Masking

File: `src/textsus/sampling/context_masking.py`. If the same recent-context window recurs within a response -- common in short or repetitive generations -- re-applying the watermark on an identical context can introduce visible repetition artifacts. This module tracks which context windows have already been used to watermark a response and skips re-biasing the token when a window repeats, falling back to plain sampling for that step instead. This corresponds to Algorithm 3 in the paper and is what makes the non-distortionary guarantee hold at the sequence level, not just per-token.

7. Scoring and Detection

7.1 Mean Score (main scorer)

File: `src/textsus/scoring/mean_score.py`. Detection needs no access to the LLM -- only the generated token ids and the watermarking key. For every token in the sequence, the seed is recomputed from the preceding tokens (mirroring generation exactly), and the g-value is computed at every tournament layer. The score is simply the mean of all these g-values. Watermarked text was generated to favor high-g-value tokens, so it scores well above the 0.5 baseline a Bernoulli(0.5) g-value gives for ordinary text; unwatermarked text hovers near 0.5.

```
def mean_score(token_ids, watermarking_key, num_layers):
    total = 0.0
    for t, token_id in enumerate(token_ids):
        seed = generate_random_seed(token_ids[:t], watermarking_key)
        for layer in range(1, num_layers + 1):
            total += generate_g_value(token_id=token_id, seed=seed, layer=layer)
    return total / (len(token_ids) * num_layers)
```

Two further scoring variants exist in the codebase, built by teammates: **`weighted_mean.py`** re-weights each tournament layer's contribution instead of averaging uniformly, and **`bayesian.py` / `frequentist.py`** convert the raw score into a posterior probability or a formal p-value respectively, rather than a bare threshold comparison.

7.2 Detector

File: `src/textsus/detection/detector.py`. Turns a score into a binary decision. *`detect_watermark`* is a plain threshold test (used throughout this project's experiments); *`detect_frequentist`* compares a p-value against a significance level alpha for the frequentist scoring path.

```
def detect_watermark(score: float, threshold: float) -> bool:
    return score >= threshold

def detect_frequentist(p_value: float, alpha: float) -> bool:
    return p_value <= alpha
```

In practice, the threshold is never hand-picked: it is calibrated empirically from unwatermarked text so that a target false-positive rate (FPR) is met, then the fraction of watermarked text scoring above that threshold is reported as the true-positive rate (TPR). This TPR-at-fixed-FPR methodology is implemented in `evaluation/metrics.py` (Section 8.1) and is the paper's own detectability metric.

8. The Generation Pipeline

File: `src/textsus/generation/watermarked_generator.py`. This is where every module above gets tied together into an actual, runnable text generator built on a HuggingFace causal LM. The *WatermarkedGenerator* class wraps a model and tokenizer, and its *generate()* method runs a standard autoregressive loop -- one forward pass per token -- but routes the next-token choice through whichever sampling method was requested.

```
class WatermarkedGenerator:
    def __init__(self, model_name, key, m=4, top_k=100, temperature=0.7,
                 gamma=0.5, delta=2.0, competitors_per_match=2, device=None):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name).to(self.device)
        ...

    def generate(self, prompt, max_new_tokens=200, watermark=True,
                 method="tournament", ignore_eos=False):
        input_ids = self.tokenizer(prompt, return_tensors="pt").input_ids
        generated = []
        for _ in range(max_new_tokens):
            logits = self.model(input_ids).logits[0, -1, :]
            probs = self._next_token_probs(logits)  # temperature + top-k

            if not watermark:
                next_token = torch.multinomial(probs, 1).item()
            elif method == "tournament":
                candidates = torch.multinomial(probs, n_candidates,
                                                replacement=True).tolist()
                seed = generate_random_seed(generated, self.key)
                next_token = (tournament_sample(candidates, seed, self.m)
                             if self.competitors_per_match == 2 else
                             distortionary_tournament_sample(candidates, seed,
                                                                self.m, self.competitors_per_match))
            elif method == "gumbel":
                next_token = gumbel_sample(probs_dict, seed)
            else: # soft_red_list
                next_token = ...  # bias logits, then sample

            generated.append(next_token)
            input_ids = torch.cat([input_ids, next_token], dim=1)
```

8.1 Key design points

- **method** selects the sampling scheme per call: "tournament" (default), "gumbel", or "soft_red_list"; `watermark=False` ignores all of this and samples plainly, giving the unwatermarked negative baseline every experiment compares against.
- **competitors_per_match** transparently switches between the team's tested, exact non-distortionary `tournament_sample()` (when equal to 2, the default) and the new generalized distortionary version (when greater than 2), so existing behavior is never disturbed by the extension.
- **ignore_eos** forces generation to run for exactly `max_new_tokens` even if the model would normally stop early, which matters for experiments that need every sample to be the same length (e.g. the length-sweep and layer-ablation experiments).
- The seed is recomputed fresh at every step from only the tokens generated so far -- exactly mirroring what the detector does later, with no extra state to keep in sync.

9. Evaluation Utilities

9.1 Detectability metric

File: `src/textsus/evaluation/metrics.py`. Implements true-positive-rate at a fixed false-positive rate (TPR@FPR), the paper's central detectability measure. The threshold is set at the (1-FPR) quantile of a set of unwatermarked scores, then TPR is the fraction of watermarked scores exceeding it.

```
def tpr_at_fpr(watermarked_scores, unwatermarked_scores, fpr=0.01):  
    threshold = np.quantile(unwatermarked_scores, 1 - fpr)  
    tpr = np.mean(np.array(watermarked_scores) > threshold)  
    return tpr, threshold
```

9.2 Perplexity

File: `src/textsus/evaluation/perplexity.py`. Computes a text's perplexity under a given model -- lower means more fluent -- used throughout the project to confirm the paper's "no significant quality loss" claim, and in the distortionary-vs-Soft-Red-List comparison as the quality-cost axis.

9.3 Self-BLEU

File: `src/textsus/evaluation/self_bleu.py`. Measures inter-response diversity: each generated text is scored against every other generated text as if they were BLEU references, and the scores are averaged. A higher Self-BLEU means responses look more similar to each other (less diverse); watermarking is expected to reduce diversity somewhat, which this metric makes measurable.

10. Experiment Scripts

Every script below lives in `experiments/`, is runnable directly with `uv run python experiments/<script>.py`, and writes its results as JSON to `results/tables/` (plus a PNG figure to `results/figures/` where applicable).

10.1 *prepare_eli5_data.py*

Downloads the sentence-transformers/eli5 dataset (a parquet-format ELI5 mirror -- the original eli5/eli5_category datasets use a loading script no longer supported by recent `datasets` versions, and eli5 itself is defunct) and writes shuffled dev/test prompt splits to `data/eli5_dev.jsonl` and `data/eli5_test.jsonl`.

10.2 *run_detectability.py*

The core end-to-end check: for each prompt, generates a watermarked and an unwatermarked response, scores both with `mean_score()`, and reports TPR@FPR -- confirming that watermarked text is reliably distinguishable from unwatermarked text.

10.3 *run_quality.py*

Generates watermarked/unwatermarked pairs and computes mean perplexity and Self-BLEU for each group, to check whether watermarking measurably hurts text quality or diversity.

10.4 *run_latency.py*

Times ms-per-token for plain generation vs. each watermarking method (warm-up run excluded), reporting percentage overhead relative to unwatermarked generation.

10.5 *run_length_temperature_sweep.py*

For each temperature, generates one long response per prompt per method once, then truncates it to several shorter lengths and rescans the score at each length -- avoiding repeated regeneration -- to show how detectability (TPR@FPR) grows with text length, compared against the Gumbel baseline.

10.6 *run_layer_ablation.py*

Varies the number of Tournament layers (m) and measures both the mean score and TPR@FPR at each setting, to show how detectability scales with layer count. Includes a `--top_k` flag, added after an initial run revealed that too small a candidate pool relative to $2^{**}m$ causes duplicate-token ties that distort the trend (see Section 11).

10.7 *run_distortionary_comparison.py*

Sweeps Tournament's `competitors_per_match` and Soft Red List's `delta`, computing TPR@FPR and mean log-perplexity at each setting for both, to directly compare their detectability-vs-quality trade-offs -- reproducing the paper's Fig. 3c-style comparison.

11. A Worked Example: Diagnosing an Unexpected Result

This section documents a real debugging example from the project, since it demonstrates how the pipeline's pieces interact in practice. The first layer-ablation run (`top_k=100`, the generator's default) showed *mean watermark score decreasing* as the number of tournament layers increased -- the opposite of the paper's expected plateau.

Root cause: the generator draws $N = 2^m$ candidate tokens with replacement from a pool truncated to the top *top_k* most likely tokens. At $m=8$, $N=256$ candidates are drawn from a pool of only 100 possible values -- by the pigeonhole principle, most draws are duplicates. When a tournament match pits a token against an identical copy of itself, the g-value comparison is a guaranteed tie with zero discriminative power, dragging the average g-value toward the uninformative baseline of 0.5 rather than the ~ 0.75 a genuine pairwise comparison produces.

Confirmation: rerunning with `--top_k 500` (comfortably above the maximum $N=256$ tested) produced the expected shape instead: score rising from $m=1$ to a peak around $m=2-4$, then a gentle plateau, with `TPR@FPR` climbing from 0.6 at $m=1$ to a saturated 1.0 from $m=2$ onward -- the textbook "diminishing returns" curve the paper describes.

This episode is a direct, practical illustration of something the paper states more abstractly: detectability depends on the entropy available in the candidate pool, and a hard *top_k* ceiling limits how much of that entropy genuinely exists, independent of how many tournament layers are requested.

12. Running the Project

```
uv sync                                # install all dependencies from pyproject.toml
uv run pytest -v                       # run the full unit test suite

uv run python experiments/prepare_eli5_data.py --n_dev 200 --n_test 200
uv run python experiments/run_detectability.py --model gpt2 --n_prompts 20
uv run python experiments/run_quality.py --model gpt2 --n_prompts 15
uv run python experiments/run_latency.py --model gpt2
uv run python experiments/run_length_temperature_sweep.py --model gpt2
uv run python experiments/run_layer_ablation.py --model gpt2 --top_k 500
uv run python experiments/run_distortionary_comparison.py --model gpt2
```

All scripts default to `gpt2` for fast, no-login iteration; pass `--model google/gemma-2b-it` (after `huggingface-cli login` and accepting Gemma's license) to match the models used in the paper.